Appendix A. Oracle-parametric machines and self-programmed execution

This appendix gives a self-contained formalization of oracle-parametric single-query machines and self-programmed execution (SPE). Relative to the classical literatures on abstract machines, oracle computation, and simulation/refinement, the main new ingredient is a *completion-generated* embedding condition: the embedded machine must be reachable from a fixed seed state in one oracle-mediated hop.

A.1 Background

Machine-based semantics and implementation models are classical. Landin's SECD-style work, Plotkin's structural operational semantics, and Gurevich's abstract state machine thesis all study how to describe computation at the right semantic level rather than only at the level of hardware execution [1, 2, 3]. The present appendix follows that tradition: the object being formalized is not a particular runtime implementation detail, but one oracle-mediated transition of an evaluator-based system.

The oracle side of the story is also classical. Turing's oracle machines and Soare's later interpretation of oracle computation as a framework for online computation provide a natural precedent for machines whose next move depends on an external response source [4, 5]. On the automata side, Lynch and Tuttle's input/output automata supply a standard model for systems that interact with an environment through explicit interface actions [6]. Our machines below are closest to these traditions, but specialized to the case where each transition contains exactly one oracle query.

The embedding notion used below is likewise adapted from standard simulation and refinement techniques. Refinement mappings and forward/backward simulations formalize the idea that one machine implements another by means of a state encoding together with a commuting transition law [7, 8]. For probabilistic systems, compare Segala and Lynch's probabilistic simulations [9]. The definitions below package the same idea in a simpler, injective, pointwise form appropriate to single-query oracle-parametric machines.

Recent AI work has also begun to describe LLM systems in state-machine terms. StateFlow explicitly models LLM workflows as state machines, while recent preprints propose host/task lifecycle models or automata-theoretic classifications of agentic systems [13, 14, 15]. Those papers are useful for context, but they do not address the specific question studied here: when a model emits executable source that determines its own next prompt, what is the right machine model, and when can an entire target machine be generated from one seed state in one completion hop?

What is specific to the present formulation is the *completion-generated* condition below: every embedded state must be reachable from a single seed state in one oracle-mediated step. That one-hop condition is the part that is specific to self-programmed execution.

A.2 Oracle-parametric single-query machines

We fix an oracle interface $(P, C, \mathcal{M})$, where P is a prompt space, C is a completion space, and each

$$M \in \mathcal{M}$$

is a stochastic oracle

$$M : P \to \Delta(C).$$

Here $\Delta(A)$ denotes the set of probability measures on A. We suppress measurability hypotheses throughout.

Definition A.2.1 (Single-query stochastic state machine)**.** *A* single-query stochastic state machine *(SSM) over* $(P, C, \mathcal{M})$ *is a triple*

$$X = (S, p, h)$$

consisting of a state space S*, a prompt function*

$$p : S \to P,$$

and a harness function

$$h : S \times C \to S \cup \{1, \uparrow\},$$

where 1 *is a distinguished halting point and* $\uparrow$ *is a distinguished divergence point.*

For each oracle $M \in \mathcal{M}$*, the induced transition kernel*

$$T_M^X : S \to \Delta(S \cup \{1, \uparrow\})$$

is defined by

$$T_M^X(s) := h(s, -)_* M(p(s)),$$

where $h(s, -) : C \to S \cup \{1, \uparrow\}$ *denotes the map* $c \mapsto h(s, c)$ *and* $h(s, -)_*$ *denotes pushforward along that map.*

Thus each transition of an SSM records exactly one oracle invocation and one returned completion. This one-query-per-step regime is the same basic interface discipline that appears in Turing's oracle machines, in Soare's online reading of oracle computation, and in input/output automata with one external interaction per transition [4, 5, 6].

The extra outcome $\uparrow$ records deterministic divergence between one oracle response and the next boundary or halt. One could instead make h partial and work with subprobability kernels, but for the present appendix it is cleaner to totalize by adjoining an explicit divergence point, in the same spirit as standard operational-semantics treatments that distinguish finite evaluation from divergence [12].

More elaborate orchestration data — transcripts, tool results, summaries, scheduler metadata, mailboxes, or subagent descriptors — can simply be stored in the machine state. As long as prompt construction and post-completion update are deterministic once the completion is known, the architecture already fits the form of an SSM.

A.3 Uniform and completion-generated embeddings

Definition A.3.1 (State embedding)**.** *Let S' and S be state spaces. A* state embedding

$$e : S' \to S$$

is an injection. By convention, the same symbol also denotes the unique extension

$$e : S' \cup \{1, \uparrow\} \to S \cup \{1, \uparrow\}$$

with $e(1) = 1$ and $e(\uparrow) = \uparrow$.

Definition A.3.2 (Uniform embedding)**.** *Let*

$$X = (S, p, h), \qquad X' = (S', p', h')$$

be SSMs over the same interface $(P, C, \mathcal{M})$. We say that X uniformly embeds *X' if there exists a state embedding*

$$e : S' \to S$$

such that, for every $s' \in S'$,

$$p(e(s')) = p'(s')$$

and, for every $c \in C$,

$$h(e(s'), c) = e(h'(s', c)).$$

We use the term *embedding* because the witness is an injective state map into the ambient machine. In the verification literature, this is the injective, pointwise special case of a strong simulation or refinement mapping [7, 8, 9]. From the model's perspective, a prompt-preserving embedding is invisible: the model only sees the prompt presented at the current state, and the condition

$$p(e(s')) = p'(s')$$

says that the ambient machine queries the model exactly as the embedded machine would. Because the post-completion dynamics also commute with e, the model sees the same prompt sequence in the embedded run as in the original run.

Proposition A.3.3 (Kernel form)**.** *If X uniformly embeds X', then for each $M \in \mathcal{M}$ the following diagram commutes:*

$$\begin{array}{ccc} S' & \xrightarrow{T_M^{X'}} & \Delta(S' \cup \{1, \uparrow\}) \\ {\scriptstyle e}\downarrow & & \downarrow{\scriptstyle e_*} \\ S & \xrightarrow[T_M^X]{} & \Delta(S \cup \{1, \uparrow\}). \end{array}$$

Proof. For each $s' \in S'$,

$$\begin{aligned} T_M^X(e(s')) &= h(e(s'), -)_* M(p(e(s'))) \\ &= h(e(s'), -)_* M(p'(s')) \\ &= (e \circ h'(s', -))_* M(p'(s')) \\ &= e_* T_M^{X'}(s'). \end{aligned}$$

□

Definition A.3.4 (Completion-generated embedding)**.** *Let $X = (S, p, h)$ and $X' = (S', p', h')$ be SSMs over the same interface. For a chosen state $x_0 \in S$, we say that the pair*

$$(X, x_0)$$

completion-generates X' *if there exists a uniform embedding witness*

$$e : S' \to S$$

such that

$$\forall s' \in S' \ \exists c \in C \text{ with } e(s') = h(x_0, c).$$

If there exists $x_0 \in S$ such that (X, x_0) completion-generates X', we say that X completion-generates *X'.*

This one-hop requirement says that the embedded state is not merely present somewhere in the ambient machine, but reachable from one fixed state x_0 by a single oracle-mediated step. That is the formal analogue of loading a state through one completion.

Example A.3.5 (Seed states and completion generation)**.** *Let $P = \{\star\}$ and let $S = C = \mathbb{F}_2$ with constant prompt map p. For each harness below, consider the SSM*

$$X = (S, p, h).$$

Then:

(1) If $h(a, b) = a + b$, then (X, x_0) completion-generates X for every $x_0 \in S$.

(2) If $h(a, b) = ab$, then $(X, 1)$ completion-generates X, but $(X, 0)$ does not.

(3) If $h(a, b) = a$, then (X, x_0) does not completion-generate X for either $x_0 \in S$.

Indeed, in case (1) the image of $h(x_0, -)$ is all of $\mathbb{F}_2$ for every x_0; in case (2) the image of $h(1, -)$ is all of $\mathbb{F}_2$ but the image of $h(0, -)$ is $\{0\}$; and in case (3) the image of $h(x_0, -)$ is always $\{x_0\}$. In all three cases the identity map is a uniform embedding of X into itself, so case (3) shows that uniform embedding alone does not imply completion generation.

A.4 Self-programmed execution

For concreteness we model self-programmed execution as a standard CEK evaluator equipped with a thin oracle wrapper. The CEK machine supplies the ordinary deterministic evaluation dynamics, together with first-class quotation and `eval`. The SPE wrapper adds only a distinguished primitive llm and the rule for pausing at, and resuming from, calls to llm.

Definition A.4.1 (CEK evaluator)**.** *Fix a call-by-value language $\mathcal{L}$ with branching and general recursion. We use a standard CEK presentation in the sense of Felleisen–Friedman and Ager–Biernacki–Danvy–Midtgaard [10, 11]: its states are triples*

$$(e, \rho, \kappa)$$

of control component, environment, and continuation, where the control component e may be either a source expression or a returned first-class value. The machine steps by a partial deterministic transition function

$$T^{\mathcal{L}}_{\mathrm{CEK}} : \Sigma^{\mathcal{L}}_{\mathrm{CEK}} \rightharpoonup \Sigma^{\mathcal{L}}_{\mathrm{CEK}} \cup \{1\}.$$

Assume further that the first-class value domain is a set V containing a subset $Q \subseteq V$ of quotations of closed source expressions, together with a built-in form `eval`*. If $q = \ulcorner E \urcorner \in Q$ quotes a closed expression E, then evaluating* `eval` *q continues as evaluation of E. We also assume a value-quotation map*

$$\mathrm{quote} : V \to Q$$

such that, for every $v \in V$, the quotation $\mathrm{quote}(v)$ *denotes a closed value expression for v; hence evaluating* `eval` $\mathrm{quote}(v)$ *deterministically yields v without invoking* llm*. As before, all finite terminal outcomes are folded into the distinguished point* 1*.*

Definition A.4.2 (SPE wrapper over a CEK evaluator)**.** *Fix a CEK evaluator as in Definition A.4.1, and fix an oracle interface $(P, C, \mathcal{M})$ with*

$$P \subseteq V, \qquad Q \subseteq C \subseteq V.$$

An SPE wrapper *is obtained by adjoining a distinguished unary primitive* llm *to $\mathcal{L}$, yielding an extended language $\mathcal{L}^{\mathrm{llm}}$ and its CEK state space*

$$\Sigma := \Sigma^{\mathcal{L}^{\mathrm{llm}}}_{\mathrm{CEK}}.$$

We use the same quotation/evaluation mechanism for closed expressions of $\mathcal{L}^{\mathrm{llm}}$: if E is a closed term of the extended language, then its quotation $\ulcorner E \urcorner$ lies in $Q \subseteq C$, and evaluating `eval` $\ulcorner E \urcorner$ *continues as evaluation of E. Define the boundary set by*

$$B := \{(\mathrm{llm}\, q, \rho, \kappa) \in \Sigma : q \in P\}.$$

Thus a boundary state is exactly a CEK state whose control component is a call to llm *on an already-evaluated prompt value $q \in P$.*

For a boundary state $\beta = (\mathrm{llm}\, q, \rho, \kappa) \in B$, define

$$p(\beta) := q.$$

For $c \in C$, define the resumed CEK state

$$r(\beta, c) := (c, \rho, \kappa),$$

that is, the unique external return step that supplies the completion value c to the waiting continuation of β. Then define

$$h : B \times C \to B \cup \{1, \uparrow\}$$

by

$$h(\beta, c) = \begin{cases} \beta' & \text{if the ordinary deterministic CEK run from } r(\beta, c) \text{ first reaches some } \beta' \in B, \\ 1 & \text{if that run first reaches the terminal point } 1, \\ \uparrow & \text{if that run is infinite and never reaches } B \cup \{1\}. \end{cases}$$

The associated SSM is

$$X_{\mathsf{SPE}} = (B, p, h).$$

Thus the wrapper contributes exactly one externally controlled step, namely

$$(\mathrm{llm}\, q, \rho, \kappa) \longmapsto (c, \rho, \kappa),$$

after which all further computation is ordinary deterministic CEK evaluation until the next boundary, halt, or divergence [12].

A.5 Pure realizations and completion-generated universality

Definition A.5.1 (Pure CEK realization)**.** *Let $X' = (S', p', h')$ be an SSM over $(P, C, \mathcal{M})$. We say that X' is* purely realizable in the underlying CEK evaluator *if there exist:*

- *an injective encoding*
$$\mathrm{enc} : S' \to V,$$
- *a pure deterministic program* $\mathsf{Prompt}(z)$ *such that evaluating* $\mathsf{Prompt}(z)$ *with* $z = \mathrm{enc}(s')$ *yields* $p'(s')$*,*
- *a first-class result type R with constructors*
$$\mathsf{next} : V \to R, \qquad \mathsf{halt} \in R,$$
together with deterministic case analysis on R,
- *a pure deterministic program* $\mathsf{Step}(z, c)$ *such that, when* $z = \mathrm{enc}(s')$*, it yields* $\mathsf{next}(\mathrm{enc}(t'))$ *if* $h'(s', c) = t' \in S'$*, it yields* halt *if* $h'(s', c) = 1$*, and it diverges without invoking* llm *if* $h'(s', c) = \uparrow$*.*

In other words, the target prompt map p' and target harness h' are both implemented inside the deterministic CEK evaluator, after state has been encoded by enc. The only oracle call in the simulated loop is the single call to llm made by the ambient SPE wrapper. The tagged result type R is only a convenient first-class way to let the source-level loop distinguish "continue with this encoded state" from "halt"; any equivalent tagged encoding would do.

Theorem A.5.2 (Uniform seed universality of SPE)**.** *Assume $P \neq \varnothing$, and fix any prompt value $q_* \in P$. Then there exists a boundary state*

$$x_0 \in B,$$

depending only on the ambient SPE wrapper and the choice of q_*, such that for every SSM $X' = (S', p', h')$ over the same oracle interface that is purely realizable in the underlying CEK evaluator, the pair

$$(X_{\mathsf{SPE}}, x_0)$$

completion-generates X'.

Proof. Let $x_0 \in B$ be the first boundary state reached by deterministic evaluation of the fixed closed term

$$\mathsf{Seed}_* := \texttt{let}\ y = \mathrm{llm}\ q_*\ \texttt{in eval}\ y.$$

This depends only on the ambient SPE wrapper and the choice of q_*.

Now fix an arbitrary purely realizable SSM $X' = (S', p', h')$. If $S' = \varnothing$, the claim is vacuous, so assume $S' \neq \varnothing$. Fix a pure CEK realization enc, Prompt, and Step.

Define a recursive source-level wrapper $\mathsf{Loop}(z)$ that computes $\mathsf{Prompt}(z)$, invokes llm on the resulting prompt, computes $\mathsf{Step}(z, c)$ on the returned completion c, and then performs case analysis on the resulting element of R: in the case $\mathsf{next}(z')$ it recurs on z', and in the case halt it returns some fixed terminal value. This term is well formed because the language has ordinary recursion and case analysis.

For each $s' \in S'$, let

$$E_{s'} := \texttt{let}\ \ z = \texttt{eval}\ \ \text{quote}(\text{enc}(s'))\ \ \texttt{in}\ \ \mathsf{Loop}(z),$$

a closed term of $\mathcal{L}^{\text{llm}}$, and let $e(s') \in B$ be the first boundary state reached by deterministic evaluation of $E_{s'}$. Since evaluating $\texttt{eval}\ \text{quote}(\text{enc}(s'))$ deterministically yields $\text{enc}(s')$, and since Prompt is pure and terminating on encoded states, this boundary exists and satisfies

$$p(e(s')) = p'(s').$$

Now resume $e(s')$ with a completion $c \in C$. By construction, the resumed CEK run continues exactly as evaluation of $\mathsf{Loop}(z)$ with $z = \text{enc}(s')$ after the call to llm has returned c. If $h'(s', c) = \uparrow$, then $\mathsf{Step}(\text{enc}(s'), c)$ diverges, so the resumed CEK run diverges before the next boundary and hence

$$h(e(s'), c) = \uparrow .$$

If $h'(s', c) = 1$, then $\mathsf{Step}(\text{enc}(s'), c)$ yields halt, so the resumed run reaches the terminal point 1. Finally, if $h'(s', c) = t' \in S'$, then $\mathsf{Step}(\text{enc}(s'), c)$ yields $\mathsf{next}(\text{enc}(t'))$, and the wrapper continues as $\mathsf{Loop}(z)$ with $z = \text{enc}(t')$. Equivalently, it continues as deterministic evaluation of the closed term $E_{t'}$ after its initial $\texttt{let}$-binding has been discharged, so the next boundary reached is exactly $e(t')$. Therefore

$$h(e(s'), c) = e(h'(s', c))$$

for all $s' \in S'$ and $c \in C$, with the conventions $e(1) = 1$ and $e(\uparrow) = \uparrow$. Since each paused CEK state $e(s')$ stores the encoded value $\text{enc}(s')$ inside the fixed wrapper template Loop, the map e is injective because enc is injective. Thus e is a uniform embedding of X' into X_{SPE}.

For each $s' \in S'$, let

$$c_{s'} := \ulcorner E_{s'} \urcorner \in Q \subseteq C.$$

When the fixed state x_0 is resumed with completion $c_{s'}$, the unique external return step yields the CEK state in which the returned value is bound to y in the continuation of Seed_*; the next ordinary CEK step is therefore evaluation of $\texttt{eval}\ \ c_{s'}$, which continues as evaluation of the closed term $E_{s'}$. The first boundary reached by that run is exactly $e(s')$. Hence

$$h(x_0, c_{s'}) = e(s')$$

for every $s' \in S'$. Therefore the pair (X_{SPE}, x_0) completion-generates X'. Since X' was arbitrary, the same fixed x_0 works for every purely realizable target machine. □

Remark A.5.3 (Interpretation)**.** *The proof reuses a classical pattern — an injective state encoding together with a simulation/refinement-style commuting square [7, 8, 9] — and adds one SPE-specific ingredient: a fixed boundary state x_0 from which every embedded state can be reached in one completion hop. The completion supplied at x_0 is a quotation of a closed source term whose first boundary is the desired embedded state. The one-hop clause is therefore genuinely about generating the next embedded state through one returned piece of source.*

Remark A.5.4 (How this maps to Spell)**.** *Spell is an instance in which quotations are first-class values accepted by* `eval`*. A generic seed term of the form* `let y = llm ... in eval y` *therefore waits for one returned source fragment and then runs it. Under the trailing-expression discipline, that returned fragment can naturally be taken to be the next closed program in the interaction, so the usual extend-extend-extend loop is exactly a sequence of states obtained by one completion hop from such a seed configuration. This is one motivation for Spell's gating discipline, under which globally effectful operations such as model calls are activated only by explicit evaluation.*

Remark A.5.5 (Closures and reification)**.** *Closures are not a fundamental obstruction; the formal issue is only whether the first-class values used for state encoding admit quotation into closed source. If a runtime value contains hidden environment data, then* quote *must expose that data in source form before the value can be carried across the completion boundary. Closure conversion or any equivalent serialization theorem would suffice.*

References

[1] P. J. Landin. The Mechanical Evaluation of Expressions. *The Computer Journal*, 6(4):308–320, 1964.

[2] G. D. Plotkin. A Structural Approach to Operational Semantics. DAIMI FN-19, Computer Science Department, Aarhus University, 1981.

[3] Yuri Gurevich. Sequential Abstract State Machines Capture Sequential Algorithms. *ACM Transactions on Computational Logic*, 1(1):77–111, 2000.

[4] A. M. Turing. Systems of Logic Based on Ordinals. *Proceedings of the London Mathematical Society*, s2-45(1):161–228, 1939.

[5] Robert I. Soare. Turing Oracle Machines, Online Computing, and Three Displacements in Computability Theory. *Annals of Pure and Applied Logic*, 160(3):368–399, 2009.

[6] Nancy A. Lynch and Mark R. Tuttle. An Introduction to Input/Output Automata. *Formal Aspects of Computing*, 1:219–246, 1989.

[7] Martín Abadi and Leslie Lamport. The Existence of Refinement Mappings. *Theoretical Computer Science*, 82(2):253–284, 1991.

[8] Nancy A. Lynch and Frits W. Vaandrager. Forward and Backward Simulations. Part I: Untimed Systems. *Information and Computation*, 121(2):214–233, 1995.

[9] Roberto Segala and Nancy A. Lynch. Probabilistic Simulations for Probabilistic Processes. *Nordic Journal of Computing*, 2(2):250–273, 1995.

[10] Matthias Felleisen and Daniel P. Friedman. Control Operators, the SECD-Machine, and the Lambda-Calculus. In *Proceedings of the 3rd Working Conference on the Formal Description of Programming Concepts*, pages 193–219, 1986.

[11] Mads Sig Ager, Dariusz Biernacki, Olivier Danvy, and Jan Midtgaard. A Functional Correspondence Between Evaluators and Abstract Machines. In *Proceedings of the 5th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming*, pages 8–19, 2003.

[12] Xavier Leroy and Hervé Grall. Coinductive Big-Step Operational Semantics. *Information and Computation*, 207(2):284–304, 2009.

[13] Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. StateFlow: Enhancing LLM Task-Solving through State-Driven Workflows. In *Proceedings of COLM 2024*, 2024.

[14] Edoardo Allegrini, Ananth Shreekumar, and Z. Berkay Celik. Formalizing the Safety, Security, and Functional Properties of Agentic AI Systems. arXiv:2510.14133, 2025.

[15] Roham Koohestani, Ziyou Li, Anton Podkopaev, and Maliheh Izadi. Are Agents Just Automata? On the Formal Equivalence Between Agentic AI and the Chomsky Hierarchy. arXiv:2510.23487, 2025.